



AI News & Skills 项目架构 — 视频口播脚本（花叔v风格）

真人录音

100% 录屏

BGM 低音电子

花叔v · 终端实战风

~85s

小红书

~10min

B站长视频

14 素材

HTML文件

真人不入镜

纯录屏

小红书

短视频（90-120 秒，9:16）

风格：花叔v终端录屏 + 右下角小窗露脸（仅开场/结尾）。口语化旁白。

段 1 · 开场 (0:00-0:12)



露脸

— 右下角摄像头小窗 → 终端窗口，执行

`ls`

查看项目根目录 → 打开

`static/directory-tree.html`

来，给你看个东西。

打开终端，一个目录——AI News & Skills。

这是我搭的一套全自动内容工厂，一个人跑的。

长什么样？往下看。

段 2 · 架构 (0:12-0:25)



关闭摄像头

— 浏览器打开

`project-architecture-pres0.html`

slide 0（三栏架构

图），鼠标依次指三块

三块。左边 **AI News 工厂**，中间**视频发布引擎**，右边**教学网站**。
从一封邮件到小红书、B站全平台发布，一条管道走完。
核心在这——**四层流水线**。

段 3 • 核心管道 (0:25-0:45)

 `project-architecture-pres0.html` **slide 2**（四层管道图）→ 每层切对应画面：
L1 → 终端 `fetch_ai_news.py` / `web_scraper.py` ;
L2 → 终端运行 `item_matcher.py` 等待 `y/n` ;
L3 → `animated/pipeline-animation.html` + `static/card-display.html` ;
L4 → 浏览器自动上传小红书

第一层**采集**—— `fetch_ai_news.py` 拉163邮箱，Claude翻译，`web_scraper.py` 全网抓。
第二层**精选**—— `item_matcher.py` 去重、打分、挑6到8条，打印出来等你按 `y`。
第三层**渲染**—— `render_combined.py` 生成HTML卡片，**1080×1440**，Playwright 截图，FFmpeg合视频。
第四层**分发**——小红书自动上传，B站，飞书推送，Vercel部署。

四层走完，你只需要点一次 `y`。

段 4 • 两个品牌 (0:45-0:58)

 `project-architecture-pres0.html` **slide 3**（双品牌对比）→ 切换选项卡打开
`xhs_publish/template-ai-skills.html`

一个管道出两种内容。
AI News 周报，每周日自动跑，蓝绿配色，6到8条。
AI Skills 深度，手动触发，暖色白皮书风格。
设计token全在 `brands/*.json` 里，换风格不改代码。

段 5 • Skills (0:58-1:10)

 VS Code 打开 `.claude/skills/` 目录 → 展开 `xhs-content-engine.md` / `style-reference.md` → `animated/competitor-modules.html`

还有三个 **Claude Code Skill**。

一句话生成6张卡片、截图提取设计token、自动追踪竞品。

把经验写成可执行指令，换个电脑也能跑。

段 6 · 收尾 (1:10-1:25)

  露脸 - 右下角摄像头小窗恢复 → `project-architecture-pres.o.html` slide 8 (三原则) → slide 9 (结语页) + 白皮书 PDF 封面

从采集到发布，全链路一个人跑通。

这就是 **AI News & Skills**。

想搭自己的？记住三条原则就行——

生产分发放耦、视觉可插拔、知识不靠人靠系统。

评论区扣「白皮书」，领取AI入门指南。

我是小H的AI进化论，下期见。

~85s

总时长

真人

录音

100%

录屏

9 素材

HTML文件

BGM: `bgm/mixkit_132.mp3` 低音电子 音量 0.08

文件	用途	时段
<code>static/directory-tree.html</code>	项目目录树	0:00
<code>project-architecture-pres.o.html</code> slide 0	三栏架构	0:12
<code>project-architecture-pres.o.html</code> slide 2	四层管道	0:25
<code>animated/pipeline-animation.html</code>	管道流动动画	0:30
<code>static/card-display.html</code>	卡片PNG展示	0:35

project-architecture-pres0.html	slide 3	双品牌对比	0:45
xhs_publish/template-ai-skills.html		Skills卡片模板	0:50
animated/competitor-modules.html		Skills竞品模块	0:58
project-architecture-pres0.html	slide 8-9	三原则+结语	1:10

script.md (cont.) — B站长视频

B站 长视频（8-10 分钟，16:9）

风格：花叔v终端实战复盘 + 右下角小窗露脸（仅开场/结尾）。口语化，不念稿。

开场 (0:00-0:50)

  露脸 — 右下角摄像头小窗 → 打开终端 → `cd ~/Desktop/AI\ news\ \&\`
`skills/AI-news/ai-news-xiaohongshu` → `ls -la` → `static/directory-tree.html`

好，今天不讲AI新闻，讲工具本身。

这个东西叫 **AI News & Skills**——我自己搭的一套内容自动化系统。
从一封邮件到一张小红书卡片，再到一条B站视频，全自动。

已经跑了几个月了，产了几十条内容。今天把它拆开给你看。

不是什么高大上的架构，就是一个人、一台电脑、一个终端，加上 **Claude Code**，硬搭出来的。

我们边看边聊。

第1章 · 怎么看这个项目 (0:50-2:30)

🎬 ❌ 关闭摄像头 — 终端 `tree -L 2` → `static/directory-tree.html` →
`project-architecture-pres0.html` **slide 1** (三子系统解耦图)

先看目录结构。三个子系统。

第一个，`ai-news-xiaohongshu/` —— 核心工厂，所有内容从这出来。

第二个，`video-publish/` —— 视频发布管线，对接小红书和B站。

第三个，`website/` —— 教学网站，AI工作流的画布式教程。

为什么拆成三个？很简单——**生产和分发是两个节奏**。

工厂产出的是内容资产——卡片、文案、视频素材。

发布层只管平台适配——尺寸、格式、API。

中间用文件目录做接口。工厂把卡片PNG扔到 `xiaohongshu/日期/` 下面，发布层去读。至于卡片怎么来的，发布层不管。

这是第一个原则：**生产和分发解耦**。

你后面想加YouTube Shorts，只要在发布层加个适配器，工厂这边一行代码不用动。

第2章 · 4层管道，一层层拆 (2:30-5:30)

🎬 `project-architecture-pres0.html` **slide 2** (四层管道总览) → 逐层深入：
L1 → 终端开 `fetch_ai_news.py` + `web_scraper.py` 代码片段；
L2 → 终端运行 `item_matcher.py`，等待 `y/n` 确认画面；
L3 → `static/card-display.html` + `animated/pipeline-animation.html`；
L4 → 浏览器自动登录小红书创作者中心

重点拆核心工厂。四层管道，一层一层来。

第一层，采集。

入口是这个——`fetch_ai_news.py`。一个Python脚本。

干的第一件事，连163邮箱IMAP，搜标题含 "AINews" 的邮件，近7天的。
拉下来之后调 **Claude API** 做中英互译。翻译不是终点，是种子——翻译完的内容给后面精选用。

同时跑全网抓取。 `web_scraper.py` ，接了5个源——**Bing搜索、Hacker News**
Algolia API、Reddit JSON、Twitter Nitter镜像、ArXiv Atom XML。
还有个 `github_trending.py` ，调 **GitHub Search API**，拉24小时star增速最快的仓库。

这层的思路很简单：**邮件是锚点，但不限于邮件**。邮件告诉你"发生了什么"，全网抓取补上"还发生了什么"。

你看，这就是采集层跑起来的样子——几条命令下去，信息全进来了。

第二层，精选。

`item_matcher.py` 接管。把所有来源——邮件翻译、网页抓取、GitHub——一起扔给Claude。

Claude做三件事：**去重、打分、挑出6到8条**。

每条都有 **quality score**，低分的自动要求重写。

然后生成多平台文案——小红书一套、B站一套、Twitter一套。

这里有个关键设计——精选结果出来之后，终端打印，等你按 **y** 或 **n**。

为什么？因为Claude的判断力在"这条值不值得发"这件事上，真的不如你。

自动化不是取代判断，是省掉90%的体力活，保留最后10%的决策权。

这个确认环节，是我刻意留的。

第三层，渲染。

`xhs_publish/` 目录。 `render_combined.py` 生成卡片HTML——**1080×1440**像素，2倍缩放。Playwright无头Chrome截图输出PNG。
同时生成双封面——小红书**3:4竖屏**，B站**16:9横屏**。

视频层—— `render_video.py` 调FFmpeg合成。每张卡片5秒，封面3秒，中间0.4秒

转场。真人录音配旁白，`bgm/` 目录里扔几首mp3，自动混入。

我觉得渲染层最容易被低估。但内容是"看起来怎么样"决定的。

所以我做了个设计——所有视觉token不在代码里，在 `brands/` 目录两个JSON文件里。颜色、字号、圆角、阴影，全是外部化配置。想换风格？改JSON，一行代码不碰。

（切换到 `brands/ai-news.json`，翻几个配置项）你看，就这样。

第四层，分发。

飞书 Webhook 推卡片消息。HTML复制到 Vercel public 目录自动部署。小红书用 Playwright 模拟浏览器登录创作者中心上传。

四层走完。从一封邮件到全平台发布，全程只点一次 y。

第3章 · 一个管道，两种内容 (5:30-7:00)

🖥️ 浏览器分屏 → 左：AI News卡片 (`static/card-display.html` + `static/caption-comparison.html`) → 右： `project-architecture-presol.html` slide 3 + `xhs_publish/template-ai-skills.html`

同一个管道，出两种完全不同的内容。看——

AI News 周报——默认模式，每周日自动跑。

蓝绿配色，**Noto Sans SC** 字体。3层 info-card——**发生了什么、深入了解、为什么值得关注**。再加一个 **Insight Box** 收尾。

6到8条新闻加3个GitHub项目，固定的。

文案路线是"专业但轻松，小白友好，有深度但不装"。

AI Skills 深度——手动触发，不定期。

暖色调，**Inter** 字体，白皮书风格。4到5张长图文，layout 是 **steps + insight-box**。文案走教学引导，有步骤有案例，不卖焦虑。

为什么需要两个品牌？因为内容节奏不一样。

周报解决**信息密度**——一周过去了，你该知道这6件事。

深度解决**理解密度**——一个话题，你得真正搞懂。

品牌配置的存在，让同一个管道同时跑两种内容还不打架。

这是第二个原则：**内容结构不变，视觉表现可插拔。**

第4章 · 三个Claude Code Skill (7:00-8:30)

```
VS Code 打开 .claude/skills/ → 依次展开 xhs-content-engine.md /  
style-reference.md / competitive-reference.md →  
animated/competitor-modules.html
```

第三个设计亮点——**Claude Code Skills**。

在 `.claude/skills/` 目录下，三个文件。

第一个，`xhs-content-engine`。说一句「生成AI内容工厂卡片」——自动出6张。封面、从手动到自动化、引擎深度拆解、对比表、安装指南。紫色主题，1080乘1440。

第二个，`style-reference`。这个我觉得最妙——看到一张好的小红书封面，截图，说「参考这个样式」。Claude自动提取设计token——主色、强调色、字体、间距、圆角。生成HTML参考页存到 `design-refs/` 目录。下次生成卡片直接引用。设计知识不是记在脑子里，是存在文件系统里的。

第三个，`competitive-reference`。追踪5个同类开源项目的更新节奏、内容策略、视觉变化。优先级Roadmap自动排。

三个Skill的本质是一样的——**把经验变成可执行指令**。

你不在，别人也能用这套东西跑出一样的卡片。

这是第三个原则：**知识不依赖人，依赖系统。**

第5章 · 三个设计原则，回顾一下 (8:30-9:30)

[project-architecture-pres0.html](#)

slide 8 (三原则) → 每条配终端demo: ① 发布层

适配器 ②

[brands/ai-news.json](#)

③

[animated/feedback-loop.html](#)

拆完架构，回头看三条贯穿始终的原则。

第一条：生产和分发解耦。

工厂产出内容资产，发布层只做平台适配。你今天加了小红书自动发布，不影响卡片生成。明天想加YouTube Shorts，发布层加个适配器就行。

第二条：视觉表现可插拔。

颜色、字体、间距、布局——全在 [brands/*.json](#) 里。品牌切换改一个字段，不碰代码。

第三条：知识不依赖人，依赖系统。

Claude Code Skills把经验变成指令。内容偏好存 **preference** 文件。竞品追踪自动跑。工具知道你要什么，不是你每次都告诉它。

这三条加在一起，一件事就清楚了——这套系统不是"帮一个人做内容"，是"把做内容这件事，变成一套可以复制的流程"。

结语 (9:30-10:30)

露脸 — 右下角摄像头小窗恢复 → 回到终端, [ls](#) 项目目录 → 白皮书 PDF 封面 →[project-architecture-pres0.html](#)

slide 9 (结语页)

我经常说一句话——**自动化不是取代判断，是省掉90%的体力活，保留最后10%的决策权。**

翻译不用自己翻，精选不用自己搜，卡片不用自己排，视频不用自己剪。
但发哪条、不要哪条、这期封面用什么色调——这些决定还是你来做。

好的自动化，边界就在这里。它把你从重复劳动里拉出来，但把你放在决策者的位置上。

这就是 **AI News & Skills**。从一封邮件，到全平台发布。从一个人，到一个内容工

厂。

如果你想搭自己的自动化管道，把架构图存下来。不需要一模一样——理解这三条原则，你搭出来的东西会比我的更好。

对了，我整理了一份**AI入门白皮书**——从零基础到能自己搭工具，涵盖Claude Code配置、Skills搭建、自动化管线设计。评论区扣「白皮书」，我私信发你。

好，这期到这。我是**小H的AI进化论**，下期见。

~10min

总时长

7 段

章节

真人

录音

14 素材

HTML文件

BGM: [bgm/mixkit_371.mp3](#) 低音科技感 音量 0.08 | 风格: 花叔v · 终端实战 · 不念稿

文件	用途	对应章节
static/directory-tree.html	项目目录树展示	开场 / 第1章 / 结语
project-architecture-pres0.html slide 0	三栏架构总览	开场
project-architecture-pres0.html slide 1	三子系统解耦	第1章
project-architecture-pres0.html slide 2	四层管道	第2章
project-architecture-pres0.html slide 3	双品牌对比	第3章
project-architecture-pres0.html slide 5-6	Skills深度拆解	第4章
project-architecture-pres0.html slide 8	三原则	第5章
project-architecture-pres0.html slide 9	结语	结语
animated/pipeline-animation.html	管道流动动画	第2章 L3
animated/feedback-loop.html	反馈闭环	第5章
animated/competitor-modules.html	竞品模块/Skills	第4章

<code>static/card-display.html</code>	卡片PNG成品	第2章 L3 / 第3章
<code>static/caption-comparison.html</code>	文案对比	第3章
<code>xhs_publish/template-ai-skills.html</code>	AI Skills卡片模板	第3章

AI News & Skills · 花叔v风格 · 终端实战感 · 真人录音 · 真人不入镜 · 14素材文件